



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Indian Institute of Technology Hyderabad

EE2801- Digital Signal Processing Lab(DSP LAB)

Experiment -4

Professor: Kiran

Done by

KURRE VINAY (EE23BTECH11036)

QUESTION:

FIR filter Design

Verify the verilog code with analog code

FIR Filter Design Mathematics

The FIR filter is designed using the Parks-McClellan algorithm, which minimizes the maximum error between the desired frequency response and the actual frequency response. The filter's impulse response $h[n]$ is given by:

$$h[n] = \sum_{k=0}^{N-1} a_k \cos(2\pi f_k n)$$

where N is the filter order, a_k are the filter coefficients, and f_k are the frequency components. The frequency response $H(\omega)$ is:

$$H(\omega) = \sum_{n=0}^{N-1} h[n] e^{-j\omega n}$$

For a bandpass filter, the ideal frequency response is:

$$H_d(\omega) = \begin{cases} 1, & \omega_1 \leq |\omega| \leq \omega_2 \\ 0, & \text{otherwise} \end{cases}$$

where ω_1 and ω_2 are the lower and upper cutoff frequencies, respectively.

The filter's output $y[n]$ is computed by convolving the input signal $x[n]$ with the impulse response $h[n]$:

$$y[n] = \sum_{k=0}^{N-1} h[k] x[n-k]$$

This convolution is implemented in both MATLAB and Verilog to filter the input signals.

filterDesigner in MATLAB

```
function Hd = filter
```

```
%FILTER Returns a discrete-time filter object.
```

```
% MATLAB Code
```

```
% Generated by MATLAB(R) 24.2 and Signal Processing Toolbox 24.2.
```

```
% Generated on: 03-Mar-2025 19:52:09
```

```
% Equiripple Bandpass filter designed using the FIRPM function.
```

```

% All frequency values are in Hz.
Fs = 48000; % Sampling Frequency

Fstop1 = 500; % First Stopband Frequency
Fpass1 = 1500; % First Passband Frequency
Fpass2 = 8000; % Second Passband Frequency
Fstop2 = 9000; % Second Stopband Frequency
Dstop1 = 0.001; % First Stopband Attenuation
Dpass = 0.057501127785; % Passband Ripple
Dstop2 = 0.0001; % Second Stopband Attenuation
dens = 20; % Density Factor

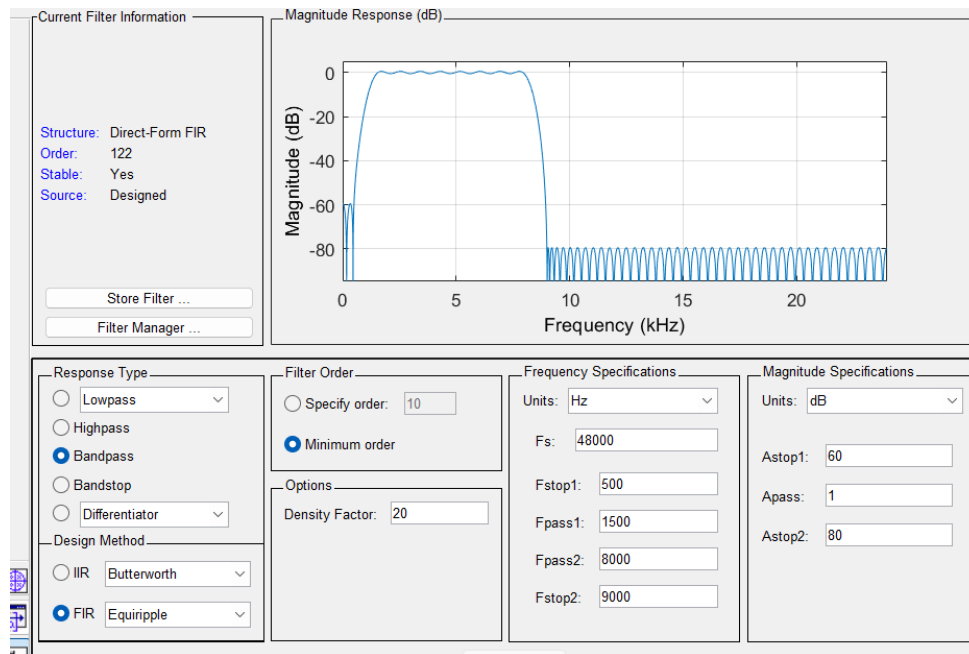
% Calculate the order from the parameters using FIRPMORD.
[N, Fo, Ao, W] = firpmord([Fstop1 Fpass1 Fpass2 Fstop2]/(Fs/2), [0 1 ...
    0], [Dstop1 Dpass Dstop2]);

% Calculate the coefficients using the FIRPM function.
b = firpm(N, Fo, Ao, W, {dens});
Hd = dfilt.dfir(b);

% [EOF]

```

Output:



MATLAB Code for fixed value and signal files

```

load('bandpass.mat');
if isempty(Num) || length(Num) < 123
error('Filter-coefficients-(Num)-are-invalid.');
```

end

```

coeffs = Num(1:123);
coeffs_fixed = fi(coeffs, 1, 16, 14);
fid = fopen('filter_coeffs.txt', 'w');
for i = 1:length(coeffs_fixed)
fprintf(fid, '%s\n', bin(coeffs_fixed(i)));
end
fclose(fid);
fs = 48000;
t = linspace(0, 5/100, fs * 5 / 100);
signals = [sin(2*pi*100*t); sin(2*pi*2000*t); sin(2*pi*6000*t); sin(2*pi*11000*t)];

if size(signals, 1) ~= 4
error('Signals-matrix-is-incorrectly-shaped.-Expected-4-rows.');
```

end

```

filter_freqs = [100, 2000, 6000, 11000];
for i = 1:4
signal_fixed = fi(signals(i, :), 1, 16, 14);
fid = fopen(sprintf('signal_%d.txt', i), 'w');
for j = 1:length(signal_fixed)
fprintf(fid, '%s\n', bin(signal_fixed(j)));
end
fclose(fid);
end
filter_freqs = [100, 2000, 6000, 11000];
disp('Script-executed-successfully.');
```

Verilog Code for FIR Filter

Verilog code

```

module fir_filter #(
    parameter COEFF_WIDTH = 16,
    parameter DATA_WIDTH = 16,
    parameter TAP_NUMBER = 20
) (
    input wire clk,
    input wire rst,
    input wire signed [DATA_WIDTH-1:0] x_in,
```

```

    output reg signed [DATA_WIDTH-1:0] y_out
);

reg signed [COEFF_WIDTH-1:0] coeffs [TAP_NUMBER-1:0];
reg signed [DATA_WIDTH-1:0] shift_reg [TAP_NUMBER-1:0];

integer i;
reg signed [DATA_WIDTH+COEFF_WIDTH-1:0] acc;

initial begin
    $readmemb("filter_coeffs_bin.txt", coeffs); // Use the binary file!
end

always @(posedge clk or posedge rst) begin
    if (rst) begin
        for (i = 0; i < TAP_NUMBER; i = i + 1) begin
            shift_reg[i] <= 0;
        end
        y_out <= 0;
    end else begin
        // Shift register
        for (i = TAP_NUMBER-1; i > 0; i = i - 1) begin
            shift_reg[i] <= shift_reg[i-1];
        end
        shift_reg[0] <= x_in;

        // Multiply and accumulate
        acc = 0;
        for (i = 0; i < TAP_NUMBER; i = i + 1) begin
            acc = acc + (shift_reg[i] * coeffs[i]);
        end

        // Output
        y_out <= acc >>> (COEFF_WIDTH-2); // Shift right to scale the output
    end
end

endmodule

```

testbench code

```

module fir_filter_tb;

    // Clock and reset signals
    reg clk, rst;
    reg signed [15:0] x_in; // Input signal
    wire signed [15:0] y_out; // Filter output

```

```

// File handling variables
integer input_file, output_file;
integer scan_file;
reg signed [15:0] captured_data;
reg [100*8:0] input_filename; // 100-character file name storage
reg [100*8:0] output_filename;

// Instantiate the FIR filter module
fir_filter uut ( // Ensure that the module name matches exactly with your FIR filter module
    .clk(clk),
    .rst(rst),
    .x_in(x_in),
    .y_out(y_out)
);

// Clock generation
always #5 clk = ~clk; // 10 ns clock period (100 MHz)

initial begin
    clk = 0;
    rst = 1; // Assert reset
    #10 rst = 0; // Deassert reset after 10 ns

    // Loop through all input signal files (signal_1Hz.txt to signal_4Hz.txt)
    for (integer i = 1; i <= 4; i = i + 1) begin
        // Construct the filename correctly using $swrite
        $swrite(input_filename, "signal_%0d.txt", i);
        $swrite(output_filename, "output_%0d.txt", i);

        // Open input file
        input_file = $fopen(input_filename, "r");
        if (input_file == 0) begin
            $display("Error: Failed to open input file-%s", input_filename);
            $finish;
        end

        // Open output file
        output_file = $fopen(output_filename, "w");
        if (output_file == 0) begin
            $display("Error: Failed to open output file-%s", output_filename);
            $finish;
        end

        // Read input data and apply the filter
        while (!$feof(input_file)) begin

```

```

        scan_file = $fscanf(input_file, "%b\n", captured_data); // Read decimal data
        from file
        if (scan_file == 1) begin
            x_in = captured_data; // Apply input to the filter
            #10; // Wait for one clock cycle
            $fwrite(output_file, "%b\n", y_out); // Write output to file
        end else begin
            $display("Warning: Failed to read data from %s", input_filename);
        end
    end

    // Close files after processing each signal
    $fclose(input_file);
    $fclose(output_file);
end

$display("Simulation completed successfully.");
$finish; // End simulation
end

endmodule

```

Comparison of verilog outputs with matlab outputs

```

Fs = 48000;
duration = 0.005;
t = 0:1/Fs:duration;
frequencies = [100, 2000, 6000, 11000];
num_frequencies = length(frequencies);
load('bandpass.mat');
coeffs = Num(1:30);
figure;
for i = 1:num_frequencies
    signal = sin(2*pi*frequencies(i)*t);
    filtered_signal_matlab = filter(coeffs, 1, signal);
    filename_verilog = sprintf('output_%d.txt', i);
    try
        fid = fopen(filename_verilog, 'r');
        if fid == -1
            error('Could not open Verilog output file: %s', filename_verilog);
        end
        verilog_output_cell = textscan(fid, '%s');
        fclose(fid);
        verilog_output_strings = verilog_output_cell{1};
    catch

```

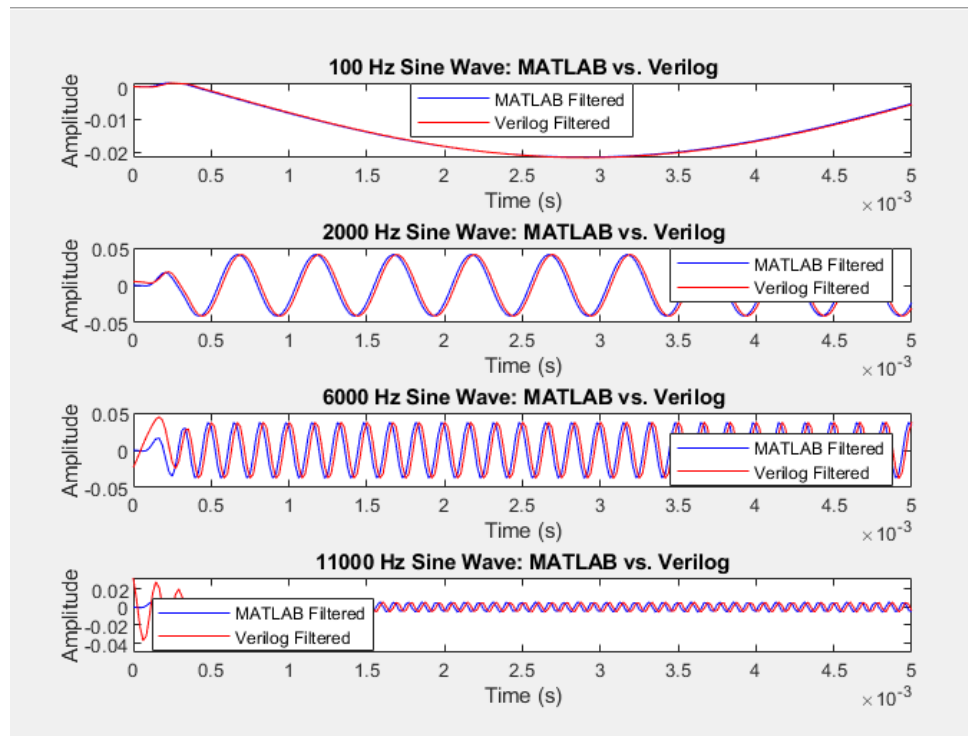
```

catch ME
    error('Error-reading-Verilog-output-file:~%s\n%s', filename_verilog, ME.message);
end
filtered_signal_verilog = zeros(size(verilog_output_strings));
for j = 1:length(verilog_output_strings)
    decimal_value = bin2dec(verilog_output_strings{j});
    if verilog_output_strings{j}(1) == '1'
        decimal_value = decimal_value - (2^16);
    end
    filtered_signal_verilog(j) = double(decimal_value) / (2^14);
end
len_verilog = length(filtered_signal_verilog);
len_matlab = length(filtered_signal_matlab);
plot_length = min(len_verilog, len_matlab);
subplot(num_frequencies, 1, i);

plot(t(1:plot_length), filtered_signal_matlab(1:plot_length), 'b', 'DisplayName', 'MATLAB-
    Filtered');
hold on;
plot(t(1:plot_length), filtered_signal_verilog(1:plot_length), 'r', 'DisplayName', 'Verilog-
    Filtered');
hold off;
title(sprintf('%d-Hz-Sine-Wave:~MATLAB-vs.-Verilog', frequencies(i)));
xlabel('Time(s)');
ylabel('Amplitude');
legend('Location', 'best');
xlim([0 duration]);
end

```

Output:

**Conculsion:**

By seeing the above plot , we can say that the verilog and matlab given almost same answers